



QNX INDIA HACKATHON | CHENNAI | SEPTEMBER 18-19, 2026

LumiGrid-RT

Safety-constrained predictive street lighting
for QNX on Raspberry Pi 4/5

FOUR PROCESSES | BOUNDED IPC | PRIORITY OVERRIDE | MEASURED EVIDENCE

Technical design, implementation guide, host test report, and target bring-up plan

PRE-EVENT LEARNING REFERENCE

Re-create and explain permitted work; replace host values with QNX/Pi evidence before submission.

Prepared September 12, 2026

Team: _____

1. Executive summary

A hardware-free software baseline that preserves the safety-critical path when real sensors are added.

LumiGrid-RT coordinates four or eight street-lighting zones using daylight, PIR occupancy, visibility, historical demand, and adjacent-zone activity. Emergency and invalid-sensor conditions always outrank energy optimization.

Selected challenge

Requirement	Implemented response	Evidence
Multiple zones	Configurable 1-8 zone controller with neighbor pre-lighting	Demo, quiet, rush, daylight, and fault scenarios
Occupancy + daylight + history	PIR/LDR simulation plus editable 24-hour profile	Telemetry and explainable probability
Emergency override	Independent service, queue priority 31, 100% target, timed clear	Integration test and CLI injection
RTOS design	Processes, fixed messages, bounded queues, FIFO requests, absolute timer	Architecture and runtime status
CLI mandatory	Interactive status, occupancy, emergency, fault, lux, reset	Live demo commands

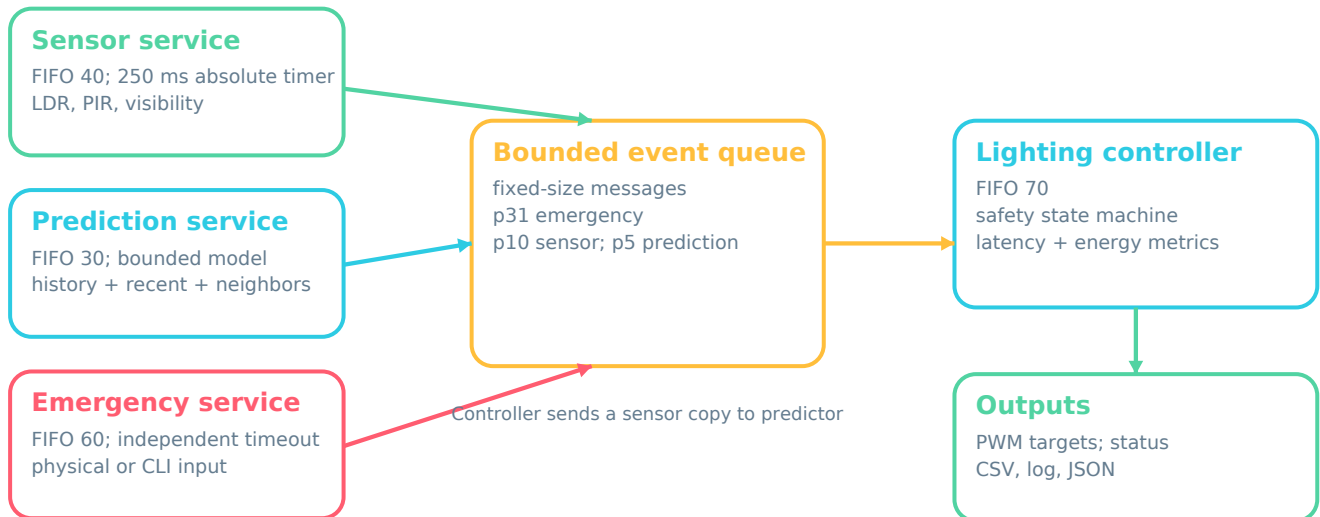
Delivery status

Completed and verified on host	Intentionally pending for Tuesday
C++17 implementation; four processes; POSIX queues; simulator; CLI; CSV/log/JSON; 18 unit assertions; integration, scenario, stress, and sanitizer runs	Licensed QNX SDP cross-build; exact Pi/BSP boot; GPIO/I2C/SPI adapter; physical emergency latency; QNX CPU/memory/priority captures

Success means safe behavior first, then measurable savings. Rush-hour savings are expected to be lower because occupied zones remain fully lit.

2. Process and IPC architecture

The predictor recommends demand; only the controller owns lamp targets.

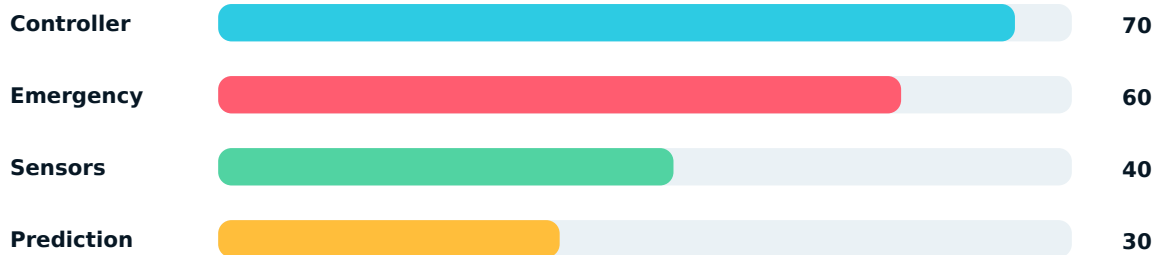


Process	FIFO	Input	Output	Failure behavior
Controller	70	Unified event queue	PWM targets, status, metrics	Uses fail-safe state if sensor data is invalid/stale
Emergency	60	Emergency command queue	Priority-31 override	Temporary command clears on independent timeout
Sensor	40	Absolute timer + commands	Priority-10 samples	Marks invalid data; never invents a valid sample
Prediction	30	Copied sensor sample	Priority-5 forecast	Controller continues from rules and sensors
CLI/supervisor	normal	Keyboard + status	Commands + clean shutdown	Outside the safety-critical path

Isolation prevents a slow predictor or display from blocking the controller. Every inter-process payload is a fixed-size trivially-copyable structure with no shared pointers.

3. Deterministic scheduling and bounded IPC

Real-time claims are designed into the software and must be re-measured on the actual QNX target.



Requested SCHED_FIFO priorities; the program reports a safe fallback if permission is denied.

Queue ordering

Message	Queue priority	Reason
Emergency override	31	Dequeue before all routine work
Clean shutdown	30	Bounded and deterministic stop
Operator command	20	Interactive control without outranking emergency
Sensor sample	10	Fresh safety input before forecasts
Prediction	5	Optimization is deliberately lowest

Absolute periodic sampling

```
deadline[n+1] = deadline[n] + period
clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &deadline, NULL);
```

- Absolute deadlines prevent cumulative timer drift when one cycle wakes late.
- A wake more than 2 ms late marks the sample cycle and increments a deadline-miss counter.
- Every queue send has a timeout; noncritical congestion drops forecast/status work rather than blocking safety work forever.
- If SCHED_FIFO permission is unavailable, the program stays functional and reports RT_FIFO=fallback instead of hiding the condition.

4. Safety state machine and prediction

The first matching state wins; daylight prevents wasted activation while emergency and fail-safe remain above it.

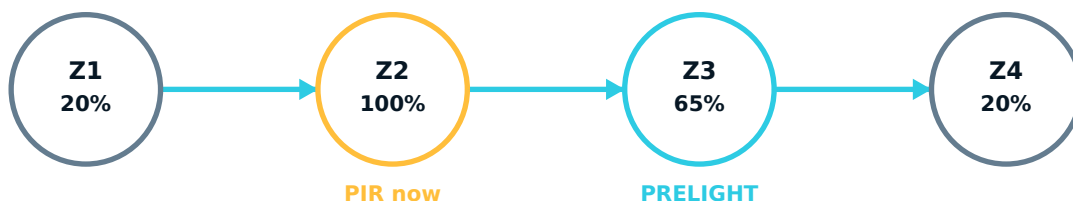
Priority	State	Trigger	Target
1	EMERGENCY	Explicit override active	100%
2	STARTUP_SAFE	No sensor sample has arrived	70%
3	SENSOR_FAILSAFE	Sample invalid or stale	70%
4	DAYLIGHT_OFF	Lux ≥ 160 and no higher state	0%
5	ACTIVE	Current or recent PIR activity	100%
6	WEATHER_SAFE	Visibility below safe threshold	75%
7	PRELIGHT	Neighbor activity or probability ≥ 0.65	65%
8	PREDICTIVE	Demand probability ≥ 0.35	40%
9	ECO	Dark road; low expected demand	20%

Brightness increases immediately for safety. Decreases ramp by 15 percentage points no faster than every 500 ms, avoiding abrupt darkness and visible flicker.

Explainable forecast

Probability = 35% recent activity + 30% historical hour + 20% adjacent zones + 15% current PIR + a bounded positive-trend bonus. The result is clamped to [0, 1].

One occupancy event lights the road ahead before arrival



5. Software package and operator interface

The simulator and hardware adapter share the same message boundary, so target integration does not rewrite policy logic.

Path	Responsibility
src/main.cpp	Queue creation, process supervision, CLI, controller runtime
src/control_engine.cpp	State precedence, dimming, energy and latency metrics
src/predictor.cpp	Historical/recent/neighbor forecast with bounded output
src/simulator.cpp	Repeatable traffic, daylight, weather, emergency, and fault scenarios
include/lumigrid/	Fixed IPC message schema and module interfaces
tests/unit_tests.cpp	Deterministic policy, prediction, and metric assertions
scripts/integration_test.sh	Full multi-process acceptance test and JSON checks
config/historical_profile.csv	Editable 24-hour traffic priors by zone

Mandatory CLI

```
$ ./lumigrid --interactive
lgrt> occupancy 2 3      # moving light wave
lgrt> emergency on 5     # timed 100% override
lgrt> fault 3 on        # sensor fail-safe
lgrt> lux 500           # daylight control
lgrt> status
lgrt> auto              # clear injected inputs
```

Recorded artifacts

Run artifact	Use in the demo/test report
telemetry.csv	Per-zone sensor, probability, target, and state timeline
events.log	Emergency, timeout, and fault transitions
summary.json	Counts, latency histogram, deadline misses, energy estimate
console.log	Human-readable runtime and CLI evidence

6. Verification method

Tests cover pure policy logic, the four-process path, deterministic scenarios, overload, and memory safety.

Layer	Method	Status
Unit	18 deterministic assertions: startup/daylight precedence, fail-safe, light wave, bounds, confidence, latency	PASS
Integration	Sensor -> controller -> predictor -> controller, timed emergency, clean shutdown, JSON assertions	PASS
Scenario matrix	Daylight, quiet night, rush hour, sensor fault as independent four-process runs	PASS
Stress	8 zones, 50 ms sampling, 10.35 seconds, 3,331 messages	PASS
Sanitizers	AddressSanitizer + UndefinedBehaviorSanitizer; leak scan disabled by container /proc restriction	PASS*
QNX target	Cross-build, priority capture, CPU/memory, 10-minute soak, physical I/O latency	PENDING

Acceptance criteria

- Emergency reaches 100% before lower-priority queued work and clears automatically when timed.
- Invalid or stale sensor data selects a nonzero 70% fail-safe.
- Occupancy gives immediate full light and pre-lights adjacent zones.
- Daylight disables lamps only when no higher-priority condition is active.
- Probability stays in [0, 1]; queue waits are bounded; all child processes stop cleanly.
- Every run emits parseable CSV, log, and JSON evidence.

Reproduce

```
make clean
make check
make demo
make interactive
```

The host runner is a portable functional and timing baseline. It is not proof of QNX scheduler performance or physical I/O latency.

7. Host validation results

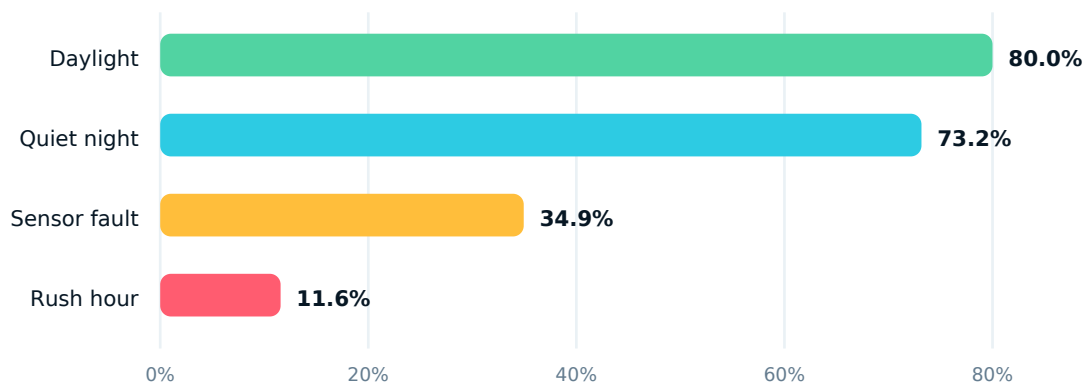
Linux 6.18 x86-64, GNU C++ 13.3.0, C++17, four zones, 100 ms scenario sampling.

HOST REFERENCE ONLY - replace timing claims with measurements from the exact QNX/Pi/BSP before final submission.

Integration reference run

Metric	Result	Metric	Result
Duration	8.300 s	Messages received	675
Sensor samples	336	Predictions	336
Emergency transitions	2	Deadline misses	0
Mean event latency	36.614 us	p95 bucket	<= 200 us
Worst event latency	243.139 us	Worst emergency	60.008 us
Estimated saving	25.619%	Stale fail-safe entries	0

Estimated energy saving by deterministic host scenario



Interpretation

- Daylight and quiet traffic produce the largest savings.
- Rush-hour savings are deliberately smaller because ACTIVE/PRELIGHT safety states dominate.
- The fault scenario enters fail-safe exactly once and never commands the failed zone dark.
- No sampling deadline was missed in these short portable runs.

8. Stress, fault, and claim boundaries

Measured behavior is only useful when the environment and limitations are stated precisely.

Eight-zone stress reference

Metric	Result	Meaning
Duration / sample period	10.350 s / 50 ms	Twice the normal sample rate at maximum configured zone count
Total messages	3,331	1,664 sensor samples + 1,664 predictions + emergency/control events
Sampling deadline misses	0	No miss observed in this short host run
p95 event-latency bucket	<= 100 us	Histogram upper bound, not an exact percentile
Worst event latency	296.222 us	Portable host measurement
Worst emergency latency	71.866 us	Queue-to-controller processing on host; excludes physical button/LED

Fault containment

Injected failure	Detection	Bounded response
Sensor invalid/stale	Validity flag + monotonic timestamp	70% SENSOR_FAILSAFE
Predictor delayed/stopped	No fresh prediction	Rules and live sensors continue
Status queue full	Timed noncritical send	One display frame is dropped
Emergency asserted	Separate service + priority-31 event	Every target goes immediately to 100%

What the host numbers do not prove

- They are not QNX scheduler measurements and do not verify actual FIFO privileges.
- They exclude Raspberry Pi GPIO/I2C/SPI driver and physical actuator delay.
- Energy is integrated commanded brightness against a 100 W-per-zone baseline, not a wattmeter reading.
- Simulated LDR values are not calibrated lux measurements.
- A longer QNX soak test is required before claiming deployment stability.

9. QNX host and target bring-up

Install only the entitlement and board support package supplied by the college/QNX coordinator.

Before Tuesday

- Confirm QNX account/license access, Software Center, SDP 8.0, Momentics, ARM64 tools, and the exact Pi 4/5 BSP.
- Open the SDP terminal and record `qcc -V`, `q++ -V`, `$QNX_HOST`, and `$QNX_TARGET`.
- Cross-build with `make qnx`; the default compiler variant is `gcc_ntoaarch64le`.
- Do not guess between Pi 4 and Pi 5 images; the BSP and boot procedure are board-specific.

```
qcc -V
q++ -V
echo "$QNX_HOST"
echo "$QNX_TARGET"
make qnx
```

Target sequence

Step	Gate	Evidence to save
1	Boot exact BSP with no sensors attached	Serial log, <code>uname -a</code> , Pi model/revision
2	Run one-file hello world	Compiler/transfer path proven
3	Run LumiGrid-RT in simulator mode on QNX	<code>summary.json</code> + console log
4	Verify mqueue and FIFO status	<code>/dev/mqueue</code> , <code>pidin ar</code> , <code>pidin threads</code>
5	Run normal/rush/emergency/fault/stress	Telemetry, latency, deadline, CPU, memory
6	Add physical devices one at a time	Pin map, driver commands, photos, physical latency

If QNX message-queue creation or real-time priority fails, preserve the exact `errno/status` and ask the mentor about the image service or abilities. Do not hide the problem by changing privileges blindly.

10. Tuesday hardware integration

Keep ControlEngine and TrafficPredictor unchanged; replace only the sensor and actuator boundaries.

Item	Qty	Purpose / note
Raspberry Pi 4 or 5 + correct BSP	1	QNX target; exact model must match the image
Power, microSD, USB-TTL	1 each	Reliable boot and recovery console
LDR + 10 kOhm resistor	1-4	Daylight divider; Pi has no analog input
ADS1115 or MCP3008	1	I2C/SPI ADC for LDR
PIR sensors	2 min	Occupancy movement across zones
LEDs + 220 Ohm resistors	4	Visible zone outputs
PCA9685 or safe drivers	1	Independent PWM; never drive high-power lamps from Pi pins
Momentary button	1	Physical emergency override
Breadboard + 3.3 V-safe jumpers	1 set	Prototype wiring with common ground

Bring-up order

- Boot QNX, then run the complete simulator on the Pi before touching GPIO.
- Confirm the BSP's GPIO/I2C/SPI resource managers and device paths.
- Toggle one safe LED; read the emergency button; read one PIR; read one LDR through ADC; then enable four PWM channels.
- Calibrate bright, covered, and corridor LDR readings; do not claim laboratory lux accuracy without a reference meter.
- Record voltage levels and never connect a 5 V output directly to a Pi GPIO.

Honest fallback: keep LDR/PIR simulated while using a physical emergency button and LEDs. This still demonstrates QNX IPC, priorities, fault handling, and real outputs if the ADC driver blocks progress.

11. Demo script and preparation schedule

The live story should prove the safety hierarchy before discussing the dashboard or aesthetics.

Five-minute judge flow

Time	Action	Proof point
0:00-0:35	Problem and promise	Save energy without sacrificing visibility
0:35-1:10	Show QNX process/queue diagram and pidin	Isolation, FIFO requests, fixed messages
1:10-2:10	Moving occupancy from Zone 1 to Zone 4	ACTIVE zone plus PRELIGHT ahead
2:10-2:50	Timed emergency	Every zone 100%; automatic timeout
2:50-3:35	Fault 3 on and low visibility	SENSOR_FAILSAFE and WEATHER_SAFE
3:35-4:20	Open summary.json	Latency, deadlines, energy, message counts
4:20-5:00	Innovation and close	Prediction is constrained; QNX protects the safety path

September 12-18 plan

Day	Primary outcome
Sat 12	Build/run baseline; learn process, thread, priority, IPC, BSP; practice CLI
Sun 13	Read and explain controller/predictor; change one threshold; add a team-owned test
Mon 14	Install QNX tools/BSP; hello world; make qnx; freeze parts list
Tue 15	Boot exact Pi image; simulator on QNX; one device at a time; capture pin/driver evidence
Wed 16	Confirmed hardware adapter; target scenario matrix; QNX CPU/memory/latency evidence
Thu 17	Freeze release candidate; replace host figures; rehearse; prepare offline backups
Fri 18	Confirm originality rules with mentor; implement permitted event work in small team commits

12. Submission integrity, risks, and references

A smaller solution that every team member can explain is stronger than a large unexplained code drop.

Originality checklist

- Treat this package as a tutor/reference. Confirm permitted pre-event material with the assigned mentor.
- Every teammate should draw the process/queue map, explain each priority, change one behavior, and update its test.
- Write official event code in small commits authored by the teammate who understands the change.
- Never label simulated readings as physical measurements or host timing as QNX real-time evidence.
- Do not publish the confidential organizer brief in the public repository.

Top risks and controls

Risk	Control
Wrong Pi/BSP or missing entitlement	Confirm exact model, license, quick-start image, and known-good SD card before Tuesday
GPIO/ADC driver delay	QNX simulator-first gate; physical emergency + LED fallback; disclose simulated inputs
Unverified real-time claims	Replace host table using pidin/Momentics and target summary.json
Demo failure	Serial/SSH fallback, copy-paste command sheet, screenshots, and recorded QNX run
Unexplained generated work	Re-create, adapt, test, document decisions, and verify mentor rules

Official technical references

QNX SDP 8.0 documentation

<https://www.qnx.com/developers/docs/8.0/>

QNX mqueue manager

<https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.utilities/topic/m/mqueue.html>

QNX clock_nanosleep()

https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.lib_ref/topic/c/clock_nanosleep.html

QNX pthread_setschedparam()

https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.lib_ref/topic/p/pthread_setschedparam.html

QNX FIFO scheduling overview

https://www.qnx.com/developers/docs/8.0/com.qnx.doc.neutrino.prog/topic/overview_FIFO_scheduling.html

QNX Software Development Platform

<https://qnx.software/products/qnx-software-development-platform/>

Source note: challenge requirements and event dates were taken from the team-supplied hackathon brief. The confidential brief is not embedded or redistributed in this public-project package.